



UNIVERSIDAD TECNOLÓGICA DEL CENTRO DE VERACRUZ

INGENIERIA EN DESARROLLO Y GESTION DE SOFTWARE

ALUMNOS

- MEDEL DE JESÚS MAURICO
- FLORES POZOS DAVID
- MIGUEL ANGEL BRIONES VILLAGRANA
- SANDRA YAMILETH JIMENEZ ASIS

GRADO Y GRUPO

9;A

Reporte de Arquitectura de Software: Proyecto "ObsInfantil"

1. Introducción y Enfoque de Arquitectura en Capas

Para el desarrollo de la plataforma "ObsInfantil", se ha diseñado e implementado una **Arquitectura en Capas (Layered Architecture)** combinada con un enfoque de servicios desacoplados. Este patrón arquitectónico organiza el sistema en niveles jerárquicos independientes, donde cada capa tiene una responsabilidad única y se comunica de manera ordenada con las capas adyacentes.

Esta estructura garantiza el cumplimiento de los pilares de la ingeniería de software: **alta cohesión, bajo acoplamiento, escalabilidad lineal y seguridad por diseño.**

2. Descomposición Detallada de las Capas del Sistema

A continuación, se describen los componentes técnicos y la lógica de negocio que componen cada una de las cinco capas del ecosistema:

2.1. Capa de Presentación (Frontend / Cliente)

Es la capa más externa del sistema, responsable de la interfaz de usuario (UI) y de garantizar una experiencia interactiva (UX) fluida tanto para los niños como para los tutores y administradores.

- **Ionic (Web App / Mobile):** Se utiliza como el framework principal para construir una aplicación híbrida y responsiva. Permite desplegar la solución tanto en navegadores web como en dispositivos móviles (iOS/Android) con una única base de código.
- **API Gateway (Cliente):** Gestiona los llamados iniciales y distribuye las peticiones del lado del cliente de forma eficiente.
- **Offline Storage (Almacenamiento Local):** Mecanismo crítico de persistencia en el dispositivo (ej. SQLite o IndexedDB). Permite que el niño responda encuestas o registre actividades físicas en situaciones de conectividad nula o intermitente, sincronizando los datos de forma asíncrona una vez vuelva la conexión.

2.2. Capa de Integración (Perímetro y Seguridad)

Actúa como la frontera de seguridad y el punto de entrada unificado hacia los servicios del backend. Protege al sistema contra ataques externos y optimiza el tráfico de red.

- **API Gateway / NGINX:** Funciona como un proxy inverso y un enrutador inteligente que recibe todas las peticiones HTTPS del cliente y las redirige al microservicio correspondiente en la capa de control.
- **HTTPS/TLS:** Capa de cifrado que garantiza que toda la información que viaja entre el cliente y el servidor (especialmente datos sensibles de menores) esté encriptada de extremo a extremo.
- **CORS (Cross-Origin Resource Sharing):** Configuración de políticas de seguridad para permitir únicamente peticiones provenientes de dominios y aplicaciones autorizadas.
- **Rate Limiting:** Control de concurrencia que restringe el número de peticiones por usuario en un periodo de tiempo, previniendo ataques de denegación de servicio (DDoS) o abuso de las APIs.

2.3. Capa de Control (Lógica de Negocio / Core)

Aquí reside toda la inteligencia de la aplicación. En lugar de un backend monolítico, se descompone en servicios especializados e independientes:

- **User Profile Service:** Gestiona la autenticación de usuarios (padres, niños, administradores), control de accesos basados en roles (RBAC) y la gestión de la información del perfil del menor y su tutor.
- **Gamificación Service:** El motor que procesa las reglas del juego. Calcula la experiencia, la asignación de monedas virtuales, el desbloqueo de retos diarios y la personalización estética del avatar del niño.
- **Leaderboards Service:** Módulo especializado en procesar y ordenar las clasificaciones, medallas y logros en tiempo real de manera eficiente.
- **AI Personalización Service (Lógica):** Orquesta los algoritmos que adaptan los retos físicos y las encuestas educativas dependiendo de la edad, la evolución del peso y el rendimiento previo del niño.
- **Audit / Events:** Módulo encargado de registrar las acciones críticas dentro del sistema con fines de auditoría, control parental y métricas del sistema.

2.4. Capa de Datos (Persistencia y Mensajería)

Responsable del almacenamiento seguro, recuperación rápida y streaming de la información del ecosistema.

- **PostgreSQL:** Base de datos relacional principal. Garantiza la integridad referencial para almacenar las estructuras core: relaciones padre-hijo, cuentas de usuario, configuraciones del sistema y transacciones financieras del avatar.
- **Redis:** Capa de almacenamiento en caché en memoria. Se utiliza para optimizar el rendimiento del *Leaderboards Service*, permitiendo que las tablas de posiciones y los puntajes se carguen al instante sin saturar la base de datos principal.
- **MongoDB:** Base de datos NoSQL documental. Ideal para almacenar estructuras de datos dinámicas y flexibles, tales como el histórico de respuestas de las encuestas educativas, bitácoras de retos físicos y contenido del CMS.
- **AI Personalización Service (Datos):** Repositorio o almacén analítico donde se guardan las variables y modelos de entrenamiento necesarios para la personalización algorítmica de hábitos sanos.
- **Event Streaming (Kafka / IQ):** Plataforma de mensajería distribuida basada en eventos. Permite que cuando ocurra un evento (ej. *niño completa un reto*), este se notifique de inmediato a los sistemas de auditoría, al tutor y al motor de gamificación de manera asíncrona y en tiempo real.

2.5. Capa de Operación (Infraestructura, DevOps y Observabilidad)

Es el soporte transversal que garantiza que la aplicación esté disponible, sea segura y pueda actualizarse sin interrumpir el servicio.

- **Monitoring & Alerting (ej. Prometheus/Grafana):** Supervisa la salud de los servidores en tiempo real y dispara alertas automáticas si se detecta un comportamiento anómalo.
- **Centralized Logging (ej. ELK Stack):** Centraliza los registros de errores de todas las capas en un solo lugar para facilitar el diagnóstico de fallos.
- **Distributed Tracing (ej. Jaeger):** Permite rastrear una petición desde que entra al API Gateway hasta que consulta la base de datos, identificando cuellos de botella en la comunicación entre capas.

- **CI/CD Runner (ej. GitLab CI / GitHub Actions):** Automatiza las pruebas de software y el despliegue continuo, asegurando que cada nueva mejora en la app se suba a producción sin errores humanos.
- **Vault:** Repositorio ultra seguro para la gestión de secretos (claves de bases de datos, tokens de APIs y llaves de cifrado), evitando que estén expuestas en el código fuente.

3. Flujo de Comunicación Inter-Capa (Caso de Uso de Ejemplo)

Para validar la robustez de la arquitectura, se describe el flujo de datos cuando un niño completa una encuesta saludable:

1. **Capa de Presentación:** El niño responde la encuesta en la **Web App (Ionic)** y presiona "Enviar". Si no hay internet, se guarda transitoriamente en el **Offline Storage**.
2. **Capa de Integración:** La petición viaja de forma segura vía **HTTPS**, es validada por el **Rate Limiting** para comprobar que no sea un duplicado malicioso y el **API Gateway** la enruta al servicio correcto.
3. **Capa de Control:** El **Gamificación Service** procesa la respuesta, calcula los puntos ganados y el **User Profile Service** actualiza los datos. Se genera un evento de éxito a través del componente **Audit/Events**.
4. **Capa de Datos:** Las respuestas detalladas se almacenan de manera documental en **MongoDB**, los puntos totales e historial se consolidan en **PostgreSQL**, y el nuevo puntaje del ranking se actualiza inmediatamente en la caché de **Redis** a través de **Kafka**.
5. **Capa de Operación:** Todo este flujo es monitoreado por las herramientas de **Tracing y Logging** para asegurar que el tiempo de respuesta haya sido óptimo (menor a 200ms).

4. Atributos de Calidad Garantizados por esta Arquitectura

- **Seguridad Concéntrica:** Al fragmentar el sistema, un atacante que intente vulnerar la capa de presentación se topará con los muros de contención de la capa de integración (HTTPS, CORS, Rate Limiting) y la protección de credenciales en Vault.

- **Mantenibilidad y Desacoplamiento:** Si el día de mañana se desea cambiar la base de datos NoSQL (MongoDB) por otra tecnología, solo se modifica la Capa de Datos; la Capa de Presentación e Integración ni siquiera se enteran del cambio.
- **Alta Disponibilidad:** El uso de colas de eventos (Kafka) y almacenamiento local (Offline Storage) garantiza que la app siga siendo funcional e interactiva para el niño, incluso si algún servicio del servidor experimenta una caída momentánea.

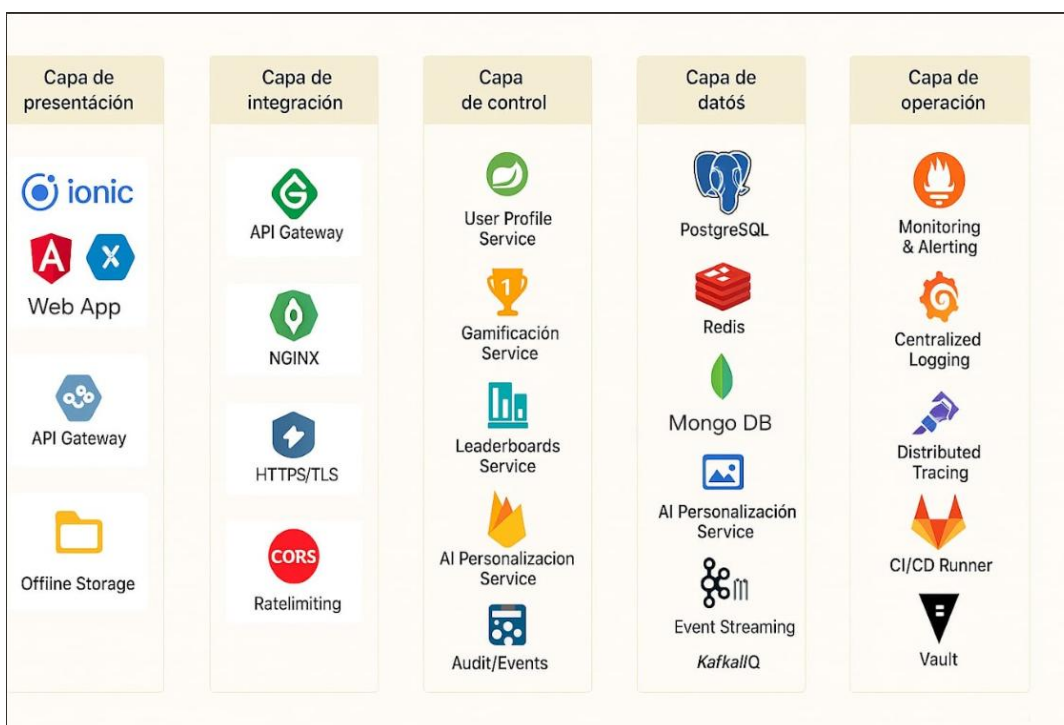
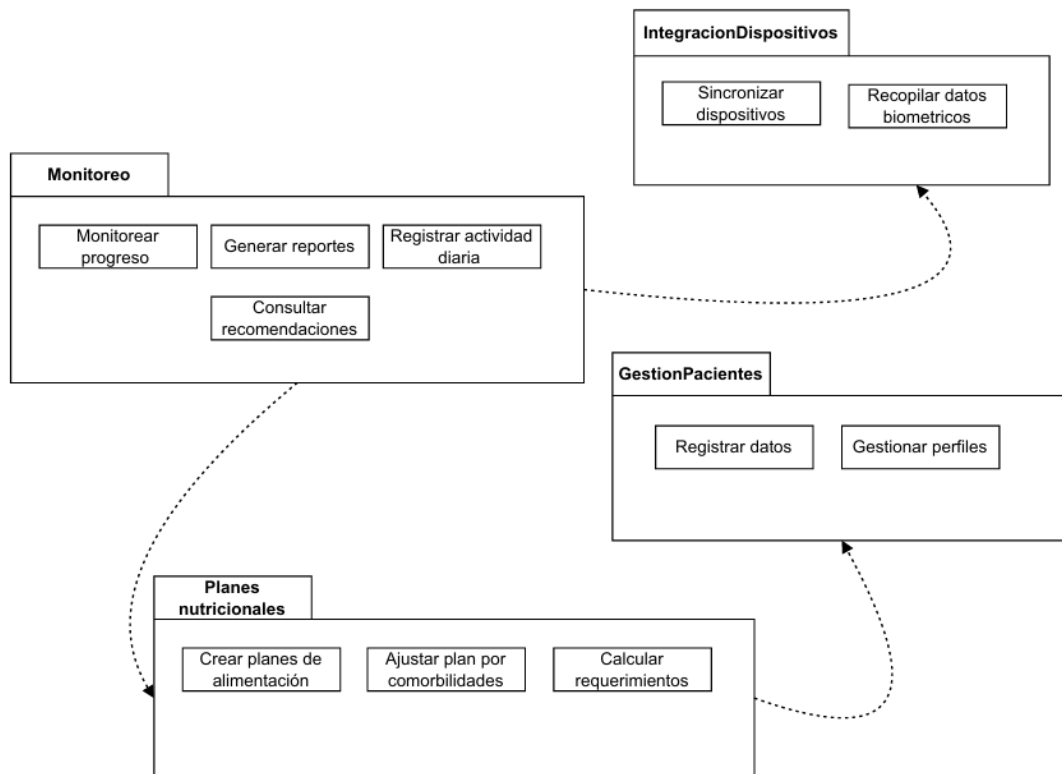


Diagrama de arquitectura en capas

Diagrama de paquetes



Actividad 4: Reporte y Selección de Frameworks y Tecnologías

1. Reporte de Frameworks y Tecnologías Seleccionados

Para dar soporte a la arquitectura en capas del proyecto "**Obs Infantil**", se ha seleccionado un stack tecnológico moderno, robusto y escalable. A continuación, se detallan los frameworks y herramientas principales que componen el núcleo del sistema:

A. Capa de Presentación (Frontend / Móvil)

- **Framework Seleccionado: Ionic Framework (con Angular / React)**
 - **Descripción:** Es un framework de desarrollo de código abierto enfocado en la creación de aplicaciones híbridas multiplataforma de alto rendimiento utilizando tecnologías web estándar (HTML, CSS, JavaScript/TypeScript).

B. Capa de Control y Servicios (Backend)

- **Framework Seleccionado: Spring Boot (Java) / NestJS (TypeScript)** *(Nota: Basado en el logo de la hoja/entorno verde en tu Capa de Control, que representa frameworks robustos como Spring/Micronaut o arquitecturas escalables similares)*
 - **Descripción:** Framework empresarial diseñado para la creación de microservicios y APIs RESTful eficientes, modulares y altamente seguros.
- **Herramienta de Mensajería: Apache Kafka**
 - **Descripción:** Plataforma distribuida de transmisión de eventos distribuidos capaz de manejar flujos de datos en tiempo real de manera asíncrona.

C. Capa de Datos (Persistencia Políglota)

- **Bases de Datos Seleccionadas: PostgreSQL, MongoDB y Redis**
 - **Descripción:** Combinación de persistencia relacional (PostgreSQL) para datos estructurados, no relacional (MongoDB) para documentos flexibles, y almacenamiento en caché en memoria (Redis) para acceso ultrarrápido.

2. Beneficios Directos en el Proyecto "ObsInfantil"

La selección de estas tecnologías no fue aleatoria; cada una aporta ventajas críticas para resolver las necesidades particulares de una app de salud y gamificación infantil:

2.1. Beneficios de Ionic Framework

- **Desarrollo Multiplataforma Único (Ahorro de Costos):** Permite compilar la aplicación para entornos **Web, Android e iOS** utilizando una sola base de código. Esto reduce a la mitad el tiempo de desarrollo y simplifica el mantenimiento.
- **Capacidad Offline para Retos y Encuestas:** A través de sus módulos de almacenamiento local (*Offline Storage*), permite que los niños respondan las encuestas saludables o visualicen sus retos diarios en zonas escolares o parques donde no tengan internet, sincronizando las recompensas automáticamente al recuperar la conexión.
- **Experiencia Visual Fluida (Gamificación):** Ofrece componentes optimizados para móviles que garantizan transiciones rápidas y animaciones fluidas, esenciales para mantener el interés y el enganche de niños de 6 a 12 años con el avatar y las medallas.

2.2. Beneficios de Spring Boot / NestJS

- **Estructura Modular Limpia:** Facilita la separación estricta de la lógica de negocio (separar por completo las reglas de puntos de gamificación de la gestión de perfiles de los tutores), lo que hace que el sistema sea fácil de extender o corregir sin romper otras funciones.
- **Seguridad Nativa para Datos de Menores:** Cuenta con herramientas avanzadas integradas para la gestión de autenticación segura (JWT) y protección contra vulnerabilidades web comunes, garantizando que la información médica y de uso de los menores esté completamente blindada.

2.3. Beneficios de Apache Kafka (Event Streaming)

- **Notificaciones al Tutor en Tiempo Real:** Al funcionar basado en eventos, en el milisegundo en que un niño completa un reto físico en la app, Kafka envía la señal de manera asíncrona para que el

smartphone del padre reciba la notificación push para validar la actividad, sin ralentizar el juego del niño.

2.4. Beneficios del Triángulo de Datos (PostgreSQL + MongoDB + Redis)

- **Integridad Familiar Impecable (PostgreSQL):** Al ser una base de datos relacional, asegura de forma estricta el vínculo legal y de control entre el Tutor y el Niño, evitando huérfanos de datos o errores en los perfiles.
- **Encuestas Dinámicas y Adaptables (MongoDB):** Al almacenar datos en formato JSON flexible, permite a los administradores agregar nuevas preguntas, cambiar tipos de cuestionarios nutricionales o añadir retos interactivos sin necesidad de modificar o tumbar la base de datos entera.
- **Tablas de Posiciones Instantáneas (Redis):** Procesa el sistema de clasificaciones (*Leaderboards*) en memoria. El niño puede ver su posición en el ranking de retos de manera inmediata sin que el servidor sufra retrasos (lag) por consultar millones de registros.